

Global Automated Mesh - Project 2

Production-Grade DevOps Pipeline & Infrastructure-as-Code Guide

1. Project Architecture Overview

This project implements a fully automated, stateful CI/CD pipeline. It transitions from local-only deployments to a production-simulated environment using AWS as the cloud provider, Terraform for infrastructure, Ansible for configuration, and K3s for orchestration.

Core Technology Stack:

- **Infrastructure:** Terraform (S3 Remote Backend)
- **Cloud:** AWS (EC2, VPC, Security Groups)
- **Configuration:** Ansible
- **Orchestration:** K3s (Kubernetes)
- **Automation:** GitHub Actions
- **Application:** Flask (Python) & PostgreSQL

2. Terraform Infrastructure Configuration

S3 Backend & Main Instance (terraform/main.tf)

```
terraform {
  backend "s3" {
    bucket = "t2k-terraform-state-bucket"
    key    = "production/terraform.tfstate"
    region = "us-west-2"
  }
}

resource "aws_instance" "mesh_worker" {
  ami                    = "ami-05d38da78ce859165"
  instance_type          = "t2.micro"
  associate_public_ip_address = true
  subnet_id              = aws_subnet.mesh_public_subnet.id
  vpc_security_group_ids = [aws_security_group.mesh_sg.id]
  key_name                = aws_key_pair.final-key.key_name

  tags = {
```

```
    Name      = "mesh-worker-node"
    Project    = "Global-Automated-Mesh"
    ManagedBy  = "Terraform"
  }
}

output "instance_public_ip" {
  value = aws_instance.mesh_worker.public_ip
}
```

3. CI/CD Workflow (GitHub Actions)

Pipeline Orchestration (.github/workflows/deploy.yml)

```
name: "Global Mesh Deployment"
on:
  push:
    branches: [main, feature/full-stack-api]

jobs:
  terraform-ansible:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: 1.7.0

      - name: Terraform Apply
        run: |
          cd terraform
          terraform init
          terraform apply -auto-approve
          # Securely capture clean IP output using JQ
          echo "NEW_IP=$(terraform output -json instance_public_ip | jq -r '.')"
      - name: Run Ansible Configuration
        env:
          ANSIBLE_HOST_KEY_CHECKING: "False"
        run: |
          echo "${{ secrets.SSH_PRIVATE_KEY }}" > ./final-key
          chmod 400 ./final-key
          ansible-playbook -i ansible/inventory.ini ansible/setup.yml
          --extra-vars "TARGET_IP=${{ env.NEW_IP }}" --private-key ./final-key -
u ubuntu
```

4. Application Logic & Deployment

Flask Connection Strategy (app/app.py)

```
import os
from flask import Flask
import psycopg2

app = Flask(__name__)

# Correctly mapped variable names to avoid 'Not Defined' errors
DB_HOST = "postgres-db"
DB_NAME = "postgres"
DB_USER = "postgres"
DB_PASSWORD = os.getenv("DB_PASSWORD", "tek2kloud-secret-pass")

@app.route('/')
def hello():
    try:
        conn = psycopg2.connect(
            host=DB_HOST, database=DB_NAME,
            user=DB_USER, password=DB_PASSWORD
        )
        return "
```

Project 2: Success!

Connected to PostgreSQL.

```
"
    except Exception as e:
        return f"
```

Project 2: Failed!

Error: {str(e)}

```
"
```

Senior DevOps Note: Ensure the Kubernetes Service matches the target NodePort (30081) and that the AWS Security Group allows TCP traffic on that range.